

System Design Knowledge Base for Building RAG Applications

Table of Contents

1. Introduction to System Design
2. Fundamentals of Distributed Systems
3. Scalability Basics
4. Load Balancing
5. Caching
6. Database Design
7. SQL Databases
8. NoSQL Databases
9. CAP Theorem
10. Data Replication
11. Database Sharding
12. Message Queues
13. Event-Driven Architecture
14. API Gateway
15. Microservices
16. Monolithic Architecture
17. Service Discovery
18. Reverse Proxy
19. CDN
20. Rate Limiting
21. Authentication
22. Authorization
23. OAuth and JWT
24. Logging and Monitoring
25. Observability
26. Distributed Tracing
27. CI/CD Pipelines
28. Docker Basics
29. Kubernetes Basics
30. Infrastructure as Code
31. Cloud Computing
32. High Availability
33. Fault Tolerance
34. Consistency Models
35. Consensus Algorithms
36. System Design Interview Framework
37. Designing URL Shortener
38. Designing YouTube
39. Designing WhatsApp

40. Designing Instagram Feed
 41. Designing Notification Systems
 42. Designing Search Systems
 43. Designing Recommendation Systems
 44. Designing Payment Systems
 45. Designing Ride Sharing Systems
 46. Designing Chat Applications
 47. Designing File Storage Systems
 48. Designing Video Streaming Platforms
 49. Designing Distributed Caches
 50. Designing Real-Time Analytics
 51. Data Engineering Concepts
 52. Data Warehousing
 53. ETL vs ELT
 54. Streaming Systems
 55. Batch Processing
 56. Big Data Systems
 57. Apache Kafka
 58. Apache Spark
 59. Elasticsearch
 60. Vector Databases
 61. Embeddings
 62. Semantic Search
 63. Retrieval-Augmented Generation
 64. Chunking Strategies
 65. Embedding Pipelines
 66. Vector Search Optimization
 67. Hybrid Search
 68. Reranking
 69. Context Windows
 70. Prompt Engineering for RAG
 71. Multi-Agent Systems
 72. AI Memory Systems
 73. Hallucination Reduction
 74. Evaluation Metrics for RAG
 75. Production RAG Architecture
 76. Security in AI Systems
 77. Observability in AI Applications
 78. Cost Optimization
 79. Latency Optimization
 80. End-to-End RAG Project Blueprint
-

1. Introduction to System Design

System design is the process of defining the architecture, components, interfaces, and data flow of a software system. It focuses on how large-scale applications are structured so they can handle millions of users, massive data volumes, and high traffic loads.

Key goals of system design:

- Scalability
- Reliability
- Availability
- Maintainability
- Security
- Performance
- Fault tolerance

Important terminology:

- Throughput: Number of requests handled per second.
- Latency: Time taken to complete a request.
- Availability: Percentage of uptime.
- Reliability: Ability to perform consistently.
- Scalability: Ability to grow with increasing traffic.

Vertical scaling means increasing machine power. Horizontal scaling means adding more machines.

Example: A startup app with 100 users may run on one server. A social media platform with 100 million users needs distributed systems.

2. Fundamentals of Distributed Systems

A distributed system is a collection of independent computers working together as one system.

Characteristics:

- Multiple machines
- Network communication
- Shared resources
- Fault tolerance

Challenges:

- Network failures
- Data consistency
- Clock synchronization

- Partial failures
- Latency

Advantages:

- Scalability
- Fault isolation
- Better performance
- Geographic distribution

Examples:

- Google Search
- Netflix
- Amazon
- WhatsApp

Important concepts:

- Nodes
 - Clusters
 - Replication
 - Partitioning
 - Consensus
-

3. Scalability Basics

Scalability measures how well a system handles growth.

Types: 1. Vertical Scaling 2. Horizontal Scaling

Vertical scaling:

- Add CPU
- Add RAM
- Easier initially
- Hardware limits exist

Horizontal scaling:

- Add more servers
- Better for large systems
- Requires load balancing
- More complex

Techniques:

- Stateless services
- Distributed caching
- Database sharding
- Queue-based processing

Bottlenecks:

- Database
 - Network
 - Disk I/O
 - CPU
 - Memory
-

4. Load Balancing

A load balancer distributes traffic across servers.

Benefits:

- Prevent overload
- Improve availability
- Increase scalability

Algorithms:

- Round Robin
- Least Connections
- IP Hash
- Weighted Round Robin

Types:

- Layer 4 load balancing
- Layer 7 load balancing

Popular tools:

- NGINX
- HAProxy
- AWS ELB

Health checks are used to detect failed servers.

5. Caching

Caching stores frequently accessed data temporarily.

Benefits:

- Lower latency
- Reduced database load
- Faster response time

Caching layers:

- Browser cache
- CDN cache
- Application cache
- Database cache

Popular caching tools:

- Redis
- Memcached

Cache eviction strategies:

- LRU
- LFU
- FIFO

Cache invalidation is one of the hardest problems in distributed systems.

6. Database Design

Database design involves structuring data for efficiency and scalability.

Goals:

- Reduce redundancy
- Ensure consistency
- Improve performance

Concepts:

- Tables
- Relationships
- Primary keys

- Foreign keys
- Indexes

Normalization reduces duplication. Denormalization improves read performance.

Tradeoffs exist between performance and consistency.

7. SQL Databases

SQL databases use structured schemas.

Examples:

- PostgreSQL
- MySQL
- Oracle
- SQL Server

Features:

- ACID transactions
- Joins
- Structured queries
- Strong consistency

Use cases:

- Banking systems
- Financial applications
- Inventory systems

ACID:

- Atomicity
 - Consistency
 - Isolation
 - Durability
-

8. NoSQL Databases

NoSQL databases are flexible and scalable.

Types:

- Document databases
- Key-value stores
- Column-family databases
- Graph databases

Examples:

- MongoDB
- Cassandra
- DynamoDB
- Neo4j

Advantages:

- Flexible schema
- Horizontal scaling
- High throughput

Tradeoffs:

- Eventual consistency
 - Complex transactions
-

9. CAP Theorem

CAP theorem states that distributed systems can guarantee only two of the following three:

- Consistency
- Availability
- Partition tolerance

CP systems prioritize consistency. AP systems prioritize availability.

Partition tolerance is mandatory in distributed systems.

Examples:

- MongoDB
 - Cassandra
 - DynamoDB
-

10. Data Replication

Replication means copying data across multiple servers.

Benefits:

- High availability
- Disaster recovery
- Faster reads

Types:

- Master-slave replication
- Multi-master replication

Challenges:

- Replication lag
 - Conflict resolution
-

11. Database Sharding

Sharding splits data across multiple databases.

Benefits:

- Horizontal scaling
- Reduced load

Sharding strategies:

- Hash-based sharding
- Range-based sharding
- Geographic sharding

Challenges:

- Rebalancing
 - Hot partitions
 - Cross-shard joins
-

12. Message Queues

Message queues enable asynchronous communication.

Benefits:

- Decoupling
- Reliability
- Scalability

Examples:

- RabbitMQ
- Kafka
- AWS SQS

Use cases:

- Email processing
 - Notifications
 - Background jobs
-

13. Event-Driven Architecture

Event-driven systems react to events.

Components:

- Producers
- Consumers
- Event brokers

Advantages:

- Loose coupling
- Scalability
- Real-time processing

Examples:

- Kafka-based systems
 - Streaming analytics
-

14. API Gateway

An API gateway acts as an entry point.

Responsibilities:

- Authentication
- Rate limiting
- Routing
- Logging

Examples:

- Kong
 - AWS API Gateway
 - NGINX
-

15. Microservices

Microservices split applications into smaller services.

Advantages:

- Independent deployment
- Better scalability
- Technology flexibility

Challenges:

- Distributed debugging
 - Network complexity
 - Service coordination
-

16. Monolithic Architecture

A monolith is a single deployable application.

Advantages:

- Simpler deployment
- Easier local development

Disadvantages:

- Hard to scale independently
 - Large codebase complexity
-

17. Service Discovery

Service discovery helps services locate each other.

Methods:

- Client-side discovery
- Server-side discovery

Tools:

- Consul
 - Eureka
 - Kubernetes DNS
-

18. Reverse Proxy

A reverse proxy forwards client requests.

Benefits:

- Security
- Load balancing
- SSL termination

Examples:

- NGINX
 - Apache
-

19. CDN

A Content Delivery Network caches content globally.

Benefits:

- Faster delivery
- Reduced origin load
- Lower latency

Examples:

- Cloudflare
 - Akamai
 - AWS CloudFront
-

20. Rate Limiting

Rate limiting controls request frequency.

Algorithms:

- Token bucket
- Leaky bucket
- Fixed window
- Sliding window

Use cases:

- API protection
 - Prevent abuse
-

21. Authentication

Authentication verifies identity.

Methods:

- Passwords
- OAuth
- SSO
- MFA

Best practices:

- Hash passwords
- Use HTTPS

- Enable MFA
-

22. Authorization

Authorization determines permissions.

Models:

- RBAC
- ABAC

Principle of least privilege is important.

23. OAuth and JWT

OAuth enables delegated access. JWT is a token format.

JWT structure:

- Header
- Payload
- Signature

Benefits:

- Stateless authentication

Risks:

- Token leakage
 - Long expiration
-

24. Logging and Monitoring

Logging captures events. Monitoring tracks system health.

Tools:

- ELK stack
- Grafana
- Prometheus

Metrics:

- CPU usage
 - Memory usage
 - Latency
 - Error rates
-

25. Observability

Observability measures internal system states.

Three pillars:

- Logs
- Metrics
- Traces

Goal:

- Faster debugging
 - Better reliability
-

26. Distributed Tracing

Tracing follows requests across services.

Tools:

- Jaeger
- Zipkin
- OpenTelemetry

Benefits:

- Root cause analysis
-

27. CI/CD Pipelines

CI/CD automates software delivery.

CI:

- Build
- Test
- Validate

CD:

- Deploy automatically

Tools:

- GitHub Actions
 - Jenkins
 - GitLab CI
-

28. Docker Basics

Docker packages applications into containers.

Benefits:

- Portability
- Isolation
- Consistency

Important terms:

- Image
 - Container
 - Dockerfile
-

29. Kubernetes Basics

Kubernetes orchestrates containers.

Core concepts:

- Pods
- Services
- Deployments
- Nodes

Benefits:

- Auto-scaling
 - Self-healing
 - Rolling updates
-

30. Infrastructure as Code

Infrastructure is managed using code.

Tools:

- Terraform
- CloudFormation
- Pulumi

Benefits:

- Reproducibility
 - Automation
-

31. Cloud Computing

Cloud providers offer scalable infrastructure.

Models:

- IaaS
- PaaS
- SaaS

Providers:

- AWS
 - Azure
 - GCP
-

32. High Availability

High availability minimizes downtime.

Strategies:

- Replication
- Failover
- Redundancy

Measured by uptime percentages.

33. Fault Tolerance

Fault tolerance allows systems to continue operating despite failures.

Methods:

- Replication
 - Retry logic
 - Circuit breakers
-

34. Consistency Models

Types:

- Strong consistency
- Eventual consistency
- Causal consistency

Tradeoffs affect performance.

35. Consensus Algorithms

Consensus ensures distributed agreement.

Algorithms:

- Paxos
- Raft

Used in:

- Distributed databases
- Kubernetes

36. System Design Interview Framework

Steps: 1. Clarify requirements 2. Estimate scale 3. Design APIs 4. Design database 5. High-level architecture 6. Deep dive 7. Identify bottlenecks 8. Optimize

37. Designing URL Shortener

Requirements:

- Generate short URLs
- Redirect quickly
- Analytics

Components:

- API service
- Database
- Cache

Key challenge:

- Unique ID generation
-

38. Designing YouTube

Requirements:

- Video upload
- Streaming
- Recommendations

Components:

- CDN
- Storage
- Encoding pipeline

Challenges:

- Massive bandwidth
 - Video transcoding
-

39. Designing WhatsApp

Requirements:

- Real-time messaging
- Delivery guarantees
- Online presence

Technologies:

- WebSockets
- Message queues

Challenges:

- Offline synchronization
 - Scalability
-

40. Designing Instagram Feed

Challenges:

- Feed ranking
- Media storage
- Infinite scrolling

Approaches:

- Fan-out on write
 - Fan-out on read
-

41. Designing Notification Systems

Types:

- Push notifications
- Email
- SMS

Key challenges:

- Reliability
- Rate limiting

- Retry mechanisms
-

42. Designing Search Systems

Search engines require:

- Crawling
- Indexing
- Ranking

Technologies:

- Elasticsearch
 - Solr
-

43. Designing Recommendation Systems

Types:

- Collaborative filtering
- Content-based filtering
- Hybrid systems

Used in:

- Netflix
 - Amazon
 - YouTube
-

44. Designing Payment Systems

Requirements:

- Security
- Idempotency
- Fraud detection

Important concepts:

- PCI compliance
- Double-entry bookkeeping

45. Designing Ride Sharing Systems

Components:

- Location tracking
- Matching engine
- Pricing engine

Challenges:

- Real-time updates
 - Geospatial indexing
-

46. Designing Chat Applications

Features:

- Presence
- Typing indicators
- Media sharing

Protocols:

- WebSockets
 - MQTT
-

47. Designing File Storage Systems

Requirements:

- Durability
- Replication
- Access control

Examples:

- Google Drive
 - Dropbox
-

48. Designing Video Streaming Platforms

Concepts:

- Adaptive bitrate streaming
- CDN caching
- Video transcoding

Protocols:

- HLS
 - DASH
-

49. Designing Distributed Caches

Goals:

- Low latency
- Horizontal scalability

Examples:

- Redis Cluster
 - Memcached
-

50. Designing Real-Time Analytics

Technologies:

- Kafka
- Spark Streaming
- Flink

Use cases:

- Fraud detection
 - Monitoring dashboards
-

51. Data Engineering Concepts

Key areas:

- Data ingestion
- Data transformation
- Data storage

Pipelines move data between systems.

52. Data Warehousing

A data warehouse stores analytical data.

Examples:

- Snowflake
- BigQuery
- Redshift

Optimized for analytical queries.

53. ETL vs ELT

ETL:

- Extract
- Transform
- Load

ELT:

- Extract
- Load
- Transform

Cloud systems often prefer ELT.

54. Streaming Systems

Streaming systems process continuous data.

Examples:

- Kafka
- Pulsar
- Kinesis

Use cases:

- Monitoring
 - Recommendation engines
-

55. Batch Processing

Batch systems process data periodically.

Examples:

- Nightly jobs
- Monthly reports

Tools:

- Hadoop
 - Spark
-

56. Big Data Systems

Big data systems manage massive datasets.

Characteristics:

- Volume
- Velocity
- Variety

Technologies:

- Hadoop
 - Spark
 - Hive
-

57. Apache Kafka

Kafka is a distributed event streaming platform.

Concepts:

- Topics
- Producers
- Consumers
- Partitions

Benefits:

- Scalability
 - Durability
-

58. Apache Spark

Spark is a distributed data processing engine.

Modules:

- Spark SQL
- Spark Streaming
- MLlib

Advantages:

- In-memory processing
-

59. Elasticsearch

Elasticsearch is a search engine.

Features:

- Full-text search
- Distributed indexing
- Aggregations

Use cases:

- Search systems
 - Log analytics
-

60. Vector Databases

Vector databases store embeddings.

Examples:

- Pinecone
- Weaviate
- Chroma
- FAISS
- Milvus

Core operations:

- Insert vectors
- Similarity search
- Metadata filtering

Distance metrics:

- Cosine similarity
 - Euclidean distance
 - Dot product
-

61. Embeddings

Embeddings convert text into numerical vectors.

Properties:

- Semantic meaning
- Dense representation

Embedding models:

- OpenAI embeddings
- Sentence Transformers
- BGE

- E5

Applications:

- Semantic search
 - Clustering
 - Recommendation systems
-

62. Semantic Search

Semantic search retrieves meaning instead of keywords.

Pipeline: 1. Convert query into embedding 2. Search vector database 3. Retrieve similar chunks

Advantages:

- Better relevance
 - Understands context
-

63. Retrieval-Augmented Generation

RAG combines retrieval systems with language models.

Architecture: 1. User query 2. Query embedding 3. Vector search 4. Retrieve documents 5. Augment prompt 6. LLM response

Benefits:

- Reduced hallucination
- Domain-specific knowledge
- Updatable knowledge base

Challenges:

- Chunking quality
 - Retrieval accuracy
 - Context limits
-

64. Chunking Strategies

Chunking divides documents into smaller pieces.

Strategies:

- Fixed-size chunking
- Recursive chunking
- Semantic chunking
- Sliding window chunking

Tradeoffs:

- Smaller chunks improve precision
- Larger chunks improve context

Overlap improves retrieval continuity.

65. Embedding Pipelines

Typical embedding pipeline: 1. Load documents 2. Clean text 3. Chunk documents 4. Generate embeddings 5. Store in vector database

Metadata examples:

- Source
 - Timestamp
 - Author
 - Topic
-

66. Vector Search Optimization

Optimization techniques:

- Approximate nearest neighbor search
- Index tuning
- Quantization
- Metadata filtering

ANN algorithms:

- HNSW
 - IVF
 - PQ
-

67. Hybrid Search

Hybrid search combines:

- Keyword search
- Semantic search

Benefits:

- Better recall
- Better precision

Common implementation:

- BM25 + vector search
-

68. Reranking

Rerankers improve search results.

Pipeline: 1. Initial retrieval 2. Rerank documents 3. Send best results to LLM

Models:

- Cross-encoders
 - Cohere rerank
 - BGE rerankers
-

69. Context Windows

LLMs have context size limits.

Challenges:

- Token overflow
- Lost context

Solutions:

- Better chunking
- Compression
- Summarization

- Retrieval filtering
-

70. Prompt Engineering for RAG

Good prompts improve answer quality.

Best practices:

- Include retrieved context
- Add clear instructions
- Use delimiters
- Ask model to cite sources

Example prompt: "Answer using only the provided context."

71. Multi-Agent Systems

AI agents collaborate to solve tasks.

Examples:

- Research agent
- Retrieval agent
- Planning agent

Frameworks:

- LangChain
 - CrewAI
 - AutoGen
-

72. AI Memory Systems

Memory systems help AI retain information.

Types:

- Short-term memory
- Long-term memory
- Vector memory

Applications:

- Personalized assistants
 - Conversational AI
-

73. Hallucination Reduction

Hallucinations are incorrect AI outputs.

Reduction techniques:

- Better retrieval
 - Grounded prompts
 - Citation enforcement
 - Fine-tuning
-

74. Evaluation Metrics for RAG

Metrics:

- Precision
- Recall
- MRR
- Faithfulness
- Context relevance

Evaluation methods:

- Human evaluation
 - Automated evaluation
-

75. Production RAG Architecture

Production RAG pipeline: 1. Data ingestion 2. Data cleaning 3. Chunking 4. Embedding generation 5. Vector storage 6. Query processing 7. Retrieval 8. Reranking 9. Prompt assembly 10. LLM generation 11. Monitoring

Important considerations:

- Caching
- Security

- Observability
 - Cost optimization
-

76. Security in AI Systems

Security risks:

- Prompt injection
- Data leakage
- Jailbreaking

Mitigations:

- Input sanitization
 - Role separation
 - Access controls
-

77. Observability in AI Applications

Monitor:

- Latency
- Token usage
- Retrieval quality
- Hallucinations

Tools:

- LangSmith
 - Weights & Biases
 - OpenTelemetry
-

78. Cost Optimization

Cost reduction methods:

- Smaller embedding models
- Caching
- Query batching
- Smaller context windows

Track:

- Token usage
 - Embedding storage
 - API calls
-

79. Latency Optimization

Latency sources:

- Embedding generation
- Retrieval
- LLM inference

Optimizations:

- Faster vector DB
 - Async pipelines
 - Streaming responses
 - Caching
-

80. End-to-End RAG Project Blueprint

Example architecture:

Frontend:

- React
- Next.js

Backend:

- FastAPI
- Node.js

LLM Layer:

- OpenAI
- Claude
- Local LLMs

Vector Database:

- Pinecone

- Chroma
- Weaviate

Pipeline: 1. Upload PDFs 2. Extract text 3. Chunk documents 4. Generate embeddings 5. Store embeddings 6. User asks question 7. Retrieve relevant chunks 8. Build prompt 9. Generate answer 10. Display citations

Advanced improvements:

- Hybrid search
- Reranking
- Multi-query retrieval
- Query rewriting
- Graph RAG
- Agentic RAG

Bonus: Common Interview Questions for RAG and System Design

1. What is RAG?
 2. Why use vector databases?
 3. Difference between semantic and keyword search?
 4. Explain chunking strategies.
 5. What causes hallucinations?
 6. What is hybrid search?
 7. Explain CAP theorem.
 8. Difference between SQL and NoSQL?
 9. How does Kafka work?
 10. What is eventual consistency?
 11. Explain database sharding.
 12. What is horizontal scaling?
 13. How does load balancing work?
 14. Explain CDN.
 15. Difference between monolith and microservices?
 16. How does Kubernetes help scaling?
 17. What is a reverse proxy?
 18. Explain caching strategies.
 19. What are embeddings?
 20. What are ANN algorithms?
-

Bonus: Sample RAG Tech Stack

Programming Language:

- Python

Frameworks:

- LangChain
- LlamaIndex
- Haystack

Embedding Models:

- OpenAI
- HuggingFace Transformers

Vector Databases:

- Chroma
- Pinecone
- FAISS

LLMs:

- GPT-4
- Claude
- Mistral
- Llama

Frontend:

- Streamlit
- React
- Gradio

Deployment:

- Docker
- Kubernetes
- AWS

Monitoring:

- LangSmith
- Grafana
- Prometheus

Bonus: End-to-End RAG Workflow Example

User uploads PDF documents.

The ingestion service extracts raw text.

The preprocessing service cleans the text.

The chunking service divides documents into chunks.

The embedding service converts chunks into vectors.

The vector database stores vectors and metadata.

When the user asks a question:

- The query is embedded.
 - Similar vectors are retrieved.
 - Top chunks are reranked.
 - Context is added to prompt.
 - LLM generates answer.
 - Sources are displayed.
-

Bonus: Common Problems in RAG Systems

Problem: Poor retrieval Solution: Better embeddings and reranking

Problem: Hallucinations Solution: Strong grounding prompts

Problem: Slow responses Solution: Caching and ANN search

Problem: Expensive inference Solution: Smaller models and batching

Problem: Token overflow Solution: Compression and filtering

Bonus: Future of RAG

Emerging trends:

- Graph RAG
- Agentic RAG

- Multimodal RAG
- Real-time RAG
- Personalized memory systems
- Long-context LLMs

AI systems are increasingly combining:

- Retrieval
- Reasoning
- Planning
- Tool usage
- Autonomous agents

Final Notes

This document can be used as:

- A RAG dataset
- Interview preparation material
- Embedding corpus
- Semantic search knowledge base
- AI assistant grounding data
- Chunking practice material

Recommended practice: 1. Chunk this file using multiple strategies. 2. Store embeddings in a vector database. 3. Build semantic search. 4. Add reranking. 5. Connect to an LLM. 6. Evaluate retrieval quality. 7. Optimize latency and costs.

End of document.